

DOCUMENTACION TECNICA DEL SISTEMA

Sistema Inteligente de Asistencia QR

Documentacion de arquitectura, modelo de datos y pipelines del sistema

MANUAL TECNICO

Documentacion de Arquitectura

VERSION

1.0

DIRIGIDO A

Equipo tecnico institucional

FECHA DE EMISION

18 de mayo de 2026

PERIODO

Año lectivo 2026

Índice de contenidos

0. Diagramas técnicos de arquitectura	6
1. Introducción Técnica	9
2. Stack Tecnológico	10
2.1 Frontend	10
2.2 Backend	10
2.3 Servicios de plataforma	11
3. Arquitectura General	12
3.1 Vista de despliegue	12
3.2 Configuración global del backend	12
3.3 Canales de comunicación cliente-servidor	12
4. Arquitectura Del Frontend	14
4.1 Organización del código	14
4.2 Gestión de estado	14
4.3 Servicios principales del frontend	14
4.4 Navegación principal	15
5. Arquitectura Del Backend	16
5.1 Organización del código	16
5.2 Patrón arquitectónico	16
5.3 Aplicación Express	16
5.4 Tipos de funciones expuestas	16
6. Modelo De Datos Firestore	18
6.1 Colecciones principales	18
6.2 Estructura representativa	18
6.3 Desnormalización intencional	20
6.4 Identificadores y claves	20
7. Reglas De Seguridad	21
7.1 Estructura general	21
7.2 Estrategia de autorización	21
7.3 Resolución del rol	21
7.4 Acceso a aulas	21
7.5 Validaciones de status en escritura	22

7.6 Lista de administradores por email	22
8. Indices Compuestos	23
8.1 Indices criticos	23
8.2 Indices de coleccion legacy	23
9. Capa De Autenticacion	24
9.1 Flujo de inicio de sesion	24
9.2 Middleware verifyFirebaseToken	24
9.3 Middlewares de autorizacion	24
9.4 Custom claims	24
9.5 Renovacion de token	24
10. Pipeline Qr	25
10.1 Generacion del codigo QR del estudiante	25
10.2 Lectura del codigo y validaciones	25
10.3 Registro de salida	26
10.4 Sesion de asistencia	26
11. Integracion Telegram	27
11.1 Configuracion del bot	27
11.2 Generacion de codigo de vinculacion	27
11.3 Normalizacion de telefonos	27
11.4 Trigger de notificacion automatica	27
11.5 Codigos de activacion	28
12. Sincronizacion Entre Colecciones	29
12.1 Doble esquema de almacenamiento	29
12.2 Trigger syncClassroomAttendanceToRoot	29
12.3 Conversion de status	29
12.4 Idempotencia	29
13. Generacion De Reportes	30
13.1 Fuentes de datos	30
13.2 Generacion PDF (modelo SIAGIE)	30
13.3 Generacion Excel (formato UGEL 06)	30
13.4 Metricas calculadas	30
14. Analisis Con Ia	32
14.1 Pipeline en tres pasos	32

14.2 Modelo y proveedor	32
14.3 Estructura del resultado	32
14.4 Latencia artificial	32
15. Configuración Y Despliegue	33
15.1 firebase.json	33
15.2 Comandos de operación	33
15.3 Flujo de despliegue recomendado	33
16. Variables De Entorno	34
16.1 Variables del backend	34
16.2 Notas operativas	34
16.3 Secretos del cliente móvil	34
17. Entorno De Desarrollo	35
17.1 Requisitos mínimos	35
17.2 Setup inicial	35
17.3 Emulación local	35
17.4 Validación de cambios	35
18. Hallazgos Arquitectónicos	36
18.1 Coexistencia de versiones de modelos del frontend	36
18.2 Coexistencia de versiones del servicio administrativo	36
18.3 Coexistencia de notación en colecciones y estados	36
18.4 Coexistencia de esquemas legacy en Firestore	36
18.5 Token de Telegram con fallback hardcoded	36
18.6 Lista de administradores por email hardcoded	36
18.7 Fallback de reportes con colección incorrecta	37
18.8 Doble pantalla de escaneo QR	37
18.9 Rol alumno modelado pero bloqueado	37
18.10 Espera artificial en análisis IA	37
19. Riesgos Técnicos Identificados	38
19.1 Riesgos de configuración	38
19.2 Riesgos operativos	38
19.3 Riesgos de datos	38
19.4 Riesgos de seguridad	38
20. Recomendaciones De Evolución	40
20.1 Consolidación del modelo de datos	40

20.2 Deprecacion de esquemas legacy 40

20.3 Endurecimiento de seguridad 40

20.4 Mejoras de fiabilidad 40

20.5 Mejoras de performance 40

20.6 Funcionalidades futuras 41

21. Glosario Tecnico 42

22. Referencias 43

22.1 Archivos clave del repositorio 43

22.2 Documentos institucionales relacionados 43

22.3 Documentacion externa relevante 43

0 Diagramas técnicos de arquitectura

Los diagramas a continuación sintetizan los pipelines principales del sistema desde una perspectiva técnica. Cada etapa numerada corresponde a un componente o acción identificada directamente en el código fuente. Estos esquemas sirven como referencia visual rápida antes de revisar las secciones técnicas correspondientes.

Diagrama 1. Arquitectura general en capas



Diagrama 2. Pipeline de autenticación



Diagrama 3. Flujo transaccional del registro QR



Diagrama 4. Pipeline de notificacion Telegram

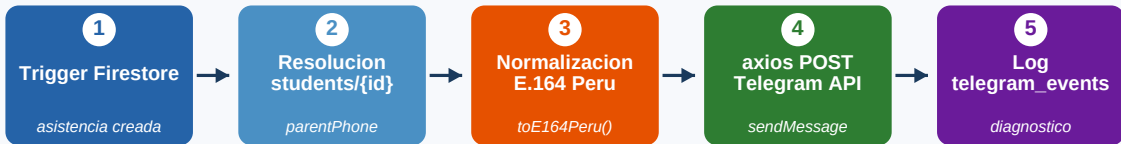
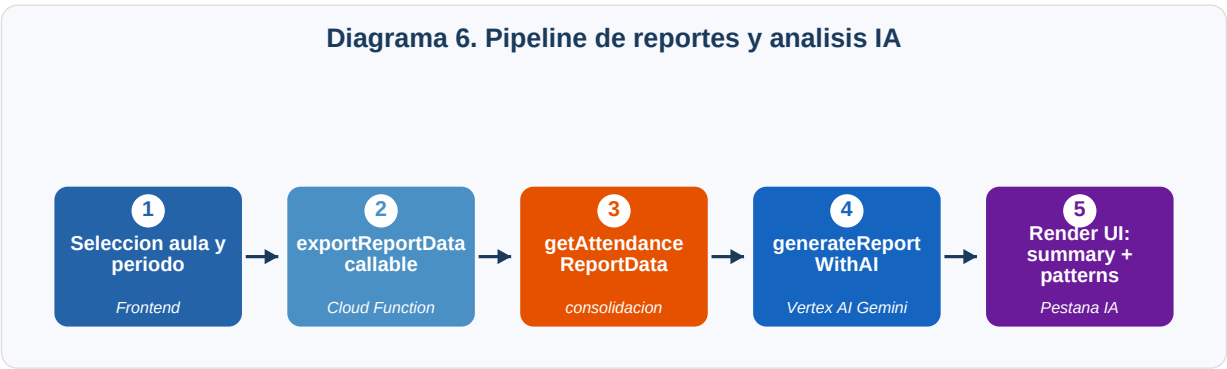


Diagrama 5. Sincronizacion dual de asistencia



Diagrama 6. Pipeline de reportes y analisis IA



1

Introducción Técnica

El presente documento describe la arquitectura, los componentes y el comportamiento operativo del **Sistema Inteligente de Asistencia QR** desde una perspectiva de ingeniería de software. Esta dirigido al equipo técnico responsable del mantenimiento, evolución y operación del sistema en producción.

El sistema está compuesto por tres bloques principales: una aplicación móvil multiplataforma desarrollada en Flutter, un backend de Cloud Functions desplegado sobre la plataforma Firebase y un conjunto de servicios de Firebase utilizados de manera nativa (Authentication, Firestore, Cloud Functions). Sobre esta base se integran dos servicios externos relevantes: la API de Telegram para notificaciones automáticas a apoderados y la plataforma Vertex AI para el módulo de análisis asistido por inteligencia artificial.

El presente manual se redacta a partir del análisis directo del código fuente, los archivos de configuración de Firebase (`firebase.json`, `firestore.rules`, `firestore.indexes.json`), los módulos del backend en `functions/src/` y los servicios del frontend en `asistencias_app/lib/`. Las referencias a archivos y comportamientos son verificables contra el repositorio del proyecto.

2 Stack Tecnológico

2.1 Frontend

Capa	Tecnología	Version
Lenguaje	Dart	SDK 3.9
Framework UI	Flutter	3.9+
Sistema de diseño	Material Design 3	nativo
Gestión de estado	Provider	6.1.2
Estado complementario	GetX	4.7.2
Navegación	GoRouter	14.2.7
Lectura de QR	mobile_scanner	5.2.3
Generación de QR	qr_flutter	4.1.0
Generación PDF on-device	pdf	3.11.1
Generación Excel on-device	excel	4.0.6
Impresión / share PDF	printing	5.13.3
Cliente HTTP	dio + http	5.7.0 / 1.2.2
Calendario	table_calendar	3.1.2
Animaciones	rive	0.13.13
Tipografías	google_fonts	6.2.1

2.2 Backend

Componente	Tecnología	Version
Runtime	Node.js	22
Lenguaje	TypeScript	5.9.2
Framework HTTP	Express	4.19.2
Plataforma serverless	Firebase Cloud Functions v2	6.0.1
SDK administrativo	firebase-admin	12.6.0
Cliente HTTP backend	axios	1.12.2
Modelo de IA	@google-cloud/vertexai	1.10.0
CORS	cors	2.8.5

2.3 Servicios de plataforma

Servicio	Uso
Firebase Authentication	Emision de JWT (Email/Password); custom claims de rol
Cloud Firestore	Base de datos NoSQL en tiempo real; transacciones atomicas
Cloud Functions	API REST, callables y triggers Firestore
Telegram Bot API	Notificaciones automaticas al apoderado
Vertex AI (Gemini)	Generacion de analisis institucional de asistencia

3 Arquitectura General

El sistema se organiza en una arquitectura cliente-servidor sobre la plataforma Firebase. El cliente móvil Flutter accede a los servicios mediante los SDKs oficiales de Firebase para autenticación, base de datos en tiempo real y llamadas a funciones serverless. La lógica de negocio sensible reside en el backend de Cloud Functions y se expone a través de dos superficies: una API REST montada sobre Express y un conjunto de funciones callable. Adicionalmente, el backend reacciona a cambios en Firestore mediante triggers para las notificaciones automáticas y la sincronización de colecciones.

3.1 Vista de despliegue

Componente	Despliegue	Region
Cloud Firestore	Firebase nativo (default database)	us-central1
Cloud Functions	Compilación TypeScript → JavaScript ejecutado en Functions v2	us-central1
Firebase Authentication	Servicio gestionado	global
Aplicación móvil	Distribución como APK / IPA	dispositivo del usuario
Telegram Bot	Servicio externo (api.telegram.org)	externo

3.2 Configuración global del backend

```
setGlobalOptions({
  maxInstances: 10,
  timeoutSeconds: 30,
  memory: '256MiB',
  region: 'us-central1'
});
```

El archivo `functions/src/index.ts` establece estos límites de manera global. Las funciones individuales pueden sobrescribir parámetros según necesidad operativa.

3.3 Canales de comunicación cliente-servidor

Tipo de operación	Mecanismo	Casos de uso
Lectura en tiempo real	Firestore Streams (<code>.snapshots()</code>)	Listas de asistencia, dashboard de estadísticas
Lectura puntual	Firestore <code>.get()</code>	Carga inicial de pantallas
Escritura atómica	Firestore Transactions (<code>.runTransaction()</code>)	Registro de asistencia QR con anti-duplicados
Llamada a función callable	SDK <code>cloud_functions</code>	Reportes IA, exportación estructurada

Tipo de operacion	Mecanismo	Casos de uso
REST API	Bearer JWT sobre HTTP a Express	Operaciones CRUD administrativas
Trigger reactivo	Firestore Trigger (onDocumentCreated / onDocumentWritten)	Notificaciones Telegram, sincronizacion dual

4 Arquitectura Del Frontend

4.1 Organización del código

La aplicación móvil se organiza bajo la siguiente estructura de directorios en `asistencias_app/lib/`:

Directorio	Responsabilidad
<code>main.dart</code>	Entry point, inicialización Firebase, AuthWrapper
<code>providers/</code>	Estado global gestionado con ChangeNotifierProvider
<code>services/</code>	Lógica de negocio, acceso a Firestore y operaciones de dominio
<code>models/</code>	Entidades del dominio y serialización
<code>screens/admin/</code>	Pantallas del rol administrador
<code>screens/auth/</code>	Flujo de autenticación
<code>screens/dashboard/</code>	Pantallas principales por rol
<code>screens/teacher/</code>	Pantallas del rol docente
<code>widgets/</code>	Componentes UI reutilizables
<code>theme/ y themes/</code>	Sistema de diseño y temas Material 3

4.2 Gestión de estado

La aplicación utiliza un modelo híbrido de gestión de estado:

- **Provider** se utiliza para el estado global de autenticación (`auth_provider.dart`) y para el estado de asistencias activas (`attendance_provider.dart`).
- **GetX** se utiliza para navegación declarativa y para utilidades reactivas locales en pantallas específicas.
- **StreamBuilder** sobre fuentes Firestore se utiliza para componentes que requieren actualización en tiempo real, como las listas de asistencia del día.

4.3 Servicios principales del frontend

Servicio	Responsabilidad
<code>auth_service.dart</code>	Login, logout, gestión de sesión Firebase Auth
<code>firestore_service.dart</code>	CRUD centralizado sobre Firestore con compatibilidad de modelos
<code>attendance_service.dart</code>	Lógica de sesión de asistencia y registro de QR

Servicio	Responsabilidad
attendance_repository.dart	Repositorio transaccional del registro QR (entrada y salida)
student_service.dart	CRUD de estudiantes y generacion de enlaces de vinculacion Telegram
classroom_service.dart	Gestion de aulas y horarios
teacher_service.dart	Operaciones del rol docente
admin_service_final.dart	Operaciones administrativas globales
academic_period_service.dart	Periodos lectivos
database_initializer.dart	Inicializacion local de cache y datos seed

4.4 Navegacion principal

La pantalla raíz `ModernDashboardScreen` implementa una navegacion por `PageView` con barra inferior de cuatro pestañas. La composicion de pestañas se determina en funcion del rol detectado en el documento `users/{uid}`:

- **Rol docente:** Inicio · Mis Aulas · Alumnos · Reportes
- **Rol administrador:** Inicio · Docentes · Estudiantes · Aulas

El componente `AuthWrapper` definido en `main.dart` se suscribe al estado de autentificacion y enruta hacia `LoginScreen` o `ModernDashboardScreen` segun corresponda. La pantalla critica para el docente es `ClassroomDetailScreen`, que integra el escaner QR embebido (`MobileScanner`), el calendario interactivo y la lista de asistencias del dia.

5 Arquitectura Del Backend

5.1 Organización del código

El backend se organiza en `functions/src/` con la siguiente estructura:

Ruta	Contenido
<code>index.ts</code>	Aplicación Express principal, middlewares de autenticación, rutas REST y configuración global
<code>telegram.ts</code>	Lógica del bot de Telegram: vinculación, notificaciones, helpers de fecha y teléfono
<code>attendance-sync.ts</code>	Trigger de sincronización entre subcolección y colección raíz de asistencia
<code>modules/admin/</code>	Rutas y operaciones administrativas (gestión de usuarios)
<code>modules/asistencias/</code>	CRUD de asistencias en arquitectura por capas (controllers, models, routes, services)
<code>modules/salones/</code>	CRUD de salones (denominación en español coexistente con <code>classrooms</code> en inglés)
<code>shared/middleware/auth.ts</code>	Middlewares de autenticación compartidos
<code>shared/types/index.ts</code>	Tipos compartidos del backend

5.2 Patrón arquitectónico

Los módulos del directorio `modules/` siguen el patrón de Clean Architecture en su forma simplificada:

- `controllers/` reciben la solicitud HTTP, validan parámetros y delegan al servicio.
- `services/` implementan la lógica de negocio y orquestan operaciones sobre Firestore.
- `models/` definen las entidades del dominio y su serialización.
- `routes/` declaran los endpoints REST y su composición con middlewares.

5.3 Aplicación Express

La aplicación Express se construye en `createApp()` dentro de `index.ts` y se exporta como una única función HTTP api a través de `onRequest`. La aplicación aplica de manera global:

- `express.json({ limit: '10mb' })` para parsing de cuerpos JSON.
- `express.urlencoded({ extended: true, limit: '10mb' })` para formularios.
- Configuración CORS abierta (`Access-Control-Allow-Origin: *`), apta para pruebas; debe restringirse en producción.
- Logging estructurado mediante `console.log` con timestamp ISO.

5.4 Tipos de funciones expuestas

Tipo	Ejemplos	Uso
HTTP (Express)	api	API REST consolidada bajo /auth/*, /estudiantes/*, /asistencias/*, /salones/*, /users/*
Callable (onCall)	createTelegramActivationLink, generateReportWithAI, getAttendanceReportData, exportReportData	Operaciones con autenticación implícita SDK
Trigger Firestore (onDocumentCreated)	Notificación Telegram al crear asistencia	Reacción a eventos de escritura
Trigger Firestore (onDocumentWritten)	syncClassroomAttendanceToRoot	Sincronización de mirror entre colecciones

6 Modelo De Datos Firestore

6.1 Colecciones principales

Coleccion	Documento	Proposito
users	{uid}	Cuentas del sistema (rol, estado, perfil)
classrooms	{classroomId}	Aulas con su configuración de horario
classrooms/{id}/attendance	{attendanceId}	Registros de asistencia por aula
classrooms/{id}/attendance_events	{eventId}	Eventos discretos entry y exit con source
students	{studentId}	Padron de estudiantes
attendance_sessions	{sessionId}	Sesiones activas de toma de asistencia
attendance	{studentId}_{date}	Mirror raíz sincronizado para queries globales
activation_codes	{autoId}	Codigos temporales de vinculacion Telegram
telegram_events	{autoId}	Log diagnostico de eventos Telegram
academic_periods	{periodId}	Periodos lectivos institucionales
salones	{salonId}	CRUD legacy en espanol (coexistente con classrooms)
asistencias	{asistenciaId}	CRUD legacy en espanol (coexistente con attendance)
whatslog	{autoId}	Log de envios de WhatsApp (escritura solo desde Functions)

6.2 Estructura representativa

users/{uid}

```
{
  uid: string,                // = Firebase Auth UID
  email: string,
  fullName: string,
  role: 'admin' | 'docente' | 'alumno',
  isActive: boolean,
  createdAt: Timestamp,
  updatedAt: Timestamp,
  profile?: { phoneNumber, department, avatarUrl }
}
```

classrooms/{classroomId}

```
{
  name: string,
  grade: string,
  section: string,
  teacherUid: string,         // FK -> users/{uid}
  teacherName?: string,      // desnormalizado
  capacity: number,
  description?: string,
  isActive: boolean,
  periodYear: number,
  schedule: {
    monday: { startTime, endTime, maxLateTime } | null,
    tuesday: ...,
    ...
    sunday: ...
  },
  createdAt: Timestamp,
  updatedAt: Timestamp
}
```

classrooms/{id}/attendance/{attendanceId}

```
{
  studentId: string,
  studentName: string,       // desnormalizado
  classroomId: string,       // redundante para queries
  sessionId: string,         // FK -> attendance_sessions
  date: string,              // 'YYYY-MM-DD'
  status: 'present' | 'late' | 'absent' | 'justified',
  timestamp: Timestamp,
  entryAt: Timestamp,
  exitAt: Timestamp | null,
  exitSource?: 'qr' | 'manual_correction',
  method?: 'qr' | 'manual',
  source?: 'qr' | 'manual' | 'auto' | 'attendance_event',
  eventDriven?: boolean,
  editedFrom?: string,       // trazabilidad de correcciones
  updatedAt?: Timestamp
}
```

students/{studentId}

```
{
  firstName: string,
  lastName: string,
  dni: string,
  classroomId: string,           // FK -> classrooms
  parentPhone?: string,         // se almacena en bruto y normalizado
  parentEmail?: string,
  qrCode: string,               // formato 'STU-{timestamp}-{random}'
  isActive: boolean,
  createdAt: Timestamp,
  updatedAt: Timestamp
}
```

attendance_sessions/{sessionId}

```
{
  classroomId: string,
  teacherUid: string,
  startTime: Timestamp,
  endTime: Timestamp | null,
  isActive: boolean,
  attendanceCount: number,
  date: string
}
```

6.3 Desnormalización intencional

El modelo de datos aplica desnormalización selectiva en los registros de asistencia (campos `studentName`, `classroomId`, `teacherName`) para tres objetivos específicos:

1. Reducir el número de lecturas necesarias por documento al renderizar listas.
2. Conservar el valor histórico de campos que pueden cambiar (por ejemplo el nombre del docente asignado al aula).
3. Permitir queries sobre la colección raíz `attendance` sin necesidad de joins.

6.4 Identificadores y claves

- Documentos de `users` usan el UID de Firebase Auth como `documentId`.
- Documentos de `attendance` raíz usan el patrón compuesto `{studentId}_{dateKey}` (ver `attendance-sync.ts:64`) para garantizar unicidad diaria.
- Códigos de activación Telegram usan IDs aleatorios autogenerados; el código de seis dígitos se almacena como campo `code`, no como `documentId`.

7 Reglas De Seguridad

7.1 Estructura general

El archivo `firestore.rules` implementa control de acceso basado en roles. Las funciones auxiliares definidas en la cabecera del archivo establecen la base de las verificaciones:

```
function isAuthed() // request.auth != null
function hasRole(role) // custom claim o documento users/{uid}
function isAdmin() // hasRole('admin')
function isDocente() // hasRole('docente') o hasRole('teacher')
function isAdminEmail() // lista blanca de correos administradores
function isDocenteOrAdmin()
function hasClassroomAccess(classroomId) // teacherUid | teacherId | ownerId | assignedClassrooms
```

7.2 Estrategia de autorizacion

El sistema implementa una **autorización en doble capa**:

1. **Firestore Security Rules** controlan el acceso directo desde el cliente a la base de datos.
2. **Middlewares del backend** (`index.ts`) controlan el acceso a la API REST y aplican validaciones adicionales contra el documento `users/{uid}`.

Esta arquitectura permite que las operaciones del cliente que ocurren sobre Firestore (lecturas en tiempo real, escrituras transaccionales) se validen en el motor de reglas, mientras que las operaciones administrativas sensibles pasan obligatoriamente por la capa REST autenticada con Bearer JWT.

7.3 Resolución del rol

Para resolver el rol del usuario, la regla `hasRole` consulta primero el custom claim `request.auth.token.role` y, si no existe, recurre al documento `users/{uid}` en Firestore. Esta resolución dual mantiene compatibilidad con cuentas creadas antes de que el sistema asignara custom claims y permite operar sobre instalaciones cuyo proceso de provisioning no incluye la asignación de claims.

7.4 Acceso a aulas

La función `hasClassroomAccess(classroomId)` autoriza al docente sobre un aula si se cumple cualquiera de los siguientes criterios:

- El documento del aula contiene `teacherUid`, `teacherId` u `ownerUid` igual al UID del usuario.
- El documento `users/{uid}` contiene un arreglo `assignedClassrooms` que incluye el `classroomId`.

Esta multiplicidad de campos refleja una migración en curso entre esquemas; el campo canónico para nuevos documentos es `teacherUid`.

7.5 Validaciones de status en escritura

Las reglas validan que el campo `status` de los registros de asistencia este contenido en el conjunto `['present', 'late', 'absent', 'presente', 'tarde', 'ausente']`. Este conjunto acepta tanto la notación canónica (en inglés) como la notación legacy (en español), lo que permite la coexistencia de registros históricos y nuevos durante la migración.

7.6 Lista de administradores por email

El archivo `firestore.rules:20` mantiene una lista blanca de correos electrónicos con privilegios de administrador (`admin@asistencias.com`, `luisangelfabian10@gmail.com`). Esta verificación (`isAdminEmail`) opera como respaldo al rol asignado en custom claims o en Firestore y debe ser mantenida con cuidado.

8 Índices Compuestos

El archivo `firestore.indexes.json` declara **31 índices compuestos** para optimizar las queries operativas del sistema. Los grupos de colección más indexados son:

Colección	Índices declarados	Propósito principal
attendance	9	Consulta por aula, por estudiante y por timestamp
classrooms	6	Listado por docente y filtrado por estado y orden alfabético
students	6	Busqueda por nombre, DNI y aula
salones	3	Compatibilidad con esquema legacy
users	2	Listado por rol y consulta por QR
asistencias	5	Compatibilidad con esquema legacy

8.1 Índices críticos

- `attendance(classroomId asc, date asc, recordedAt asc)` — listado de asistencia diaria por aula
- `attendance(studentId asc, date desc)` — historial individual del estudiante
- `attendance(classroomId asc, timestamp desc)` — feed en tiempo real ordenado
- `classrooms(teacherUid asc, isActive asc, updatedAt desc)` — aulas asignadas a un docente activas
- `students(classroomId asc, isActive asc, lastName asc, firstName asc)` — listado ordenado alfabéticamente

8.2 Índices de colección legacy

Los índices declarados sobre salones y asistencias permiten la operación de un esquema en español mantenido por compatibilidad con clientes y endpoints heredados. La eliminación de estos esquemas debe planificarse junto con la migración completa de los clientes que aun los utilicen.

9

Capa De Autenticacion

9.1 Flujo de inicio de sesion

```
[1] Cliente Flutter ejecuta signInWithEmailAndPassword(email, password)
[2] Firebase Auth valida credenciales y emite un ID Token JWT
[3] SDK de Firebase entrega el token al cliente
[4] AuthProvider escucha authStateChanges() y actualiza el estado
[5] Para llamadas REST: el token se incluye como Bearer en Authorization
[6] Para Firestore: el SDK adjunta el token automaticamente
[7] Para Cloud Functions callable: el SDK adjunta el token automaticamente
```

9.2 Middleware verifyFirebaseToken

El middleware definido en `index.ts:42-94` realiza la siguiente secuencia:

1. Extrae el token del header `Authorization: Bearer <token>`. Si esta ausente, responde con HTTP 401.
2. Verifica el token mediante `admin.auth().verifyIdToken(token)`. Si es invalido, responde 401.
3. Consulta el documento `users/{decodedToken.uid}` en Firestore. Si no existe, responde 404.
4. Verifica `userData.isActive`. Si es falso, responde 403.
5. Adjunta `req.user = { uid, email, role, fullName, isActive, ...userData }` para uso posterior.

9.3 Middlewares de autorizacion

- `requireAdmin`: verifica `req.user.role === 'admin'`. Responde 403 en caso contrario.
- `requireDocenteOrAdmin`: acepta 'docente' o 'admin'. Responde 403 en caso contrario.
- `blockStudentAccess`: rechaza con 403 cualquier solicitud cuyo rol sea 'alumno'. Esta restriccion documenta el estado beta del sistema; el rol existe en el modelo pero no opera en esta version.

9.4 Custom claims

El backend asigna custom claims al rol del usuario en el momento de la creacion de la cuenta (por ejemplo en `/auth/register-docente`). Las custom claims permiten que las reglas de Firestore resuelvan el rol sin requerir una lectura adicional del documento del usuario, lo que reduce la latencia y el costo de las operaciones.

9.5 Renovacion de token

El SDK de Firebase administra automaticamente la renovacion de los ID Tokens. El cliente movil no necesita implementar logica explicita de refresco. Los tokens emitidos por Firebase Auth tienen una validez de una hora.

10 Pipeline Qr

10.1 Generación del código QR del estudiante

El código QR se genera al momento de crear el estudiante mediante `StudentService.createStudent()`. El formato del campo `qrCode` documentado en `database_diagram.dbml:101-102` es `STU-{timestamp}-{random}`. El cliente Flutter encapsula esta cadena en un widget `QrImageView` (`qr_flutter`) para su visualización y compartición.

10.2 Lectura del código y validaciones

La pantalla `ClassroomDetailScreen` instancia un `MobileScanner` embebido. Cada lectura activa la siguiente cadena de validaciones:

- [1] Debounce físico: si el mismo QR fue leído en menos de 1200 ms, se ignora. Evita registros múltiples por movimiento de la cámara.
- [2] Decodificación: se intenta parsear como JSON. Debe contener al menos los campos `type`, `id` y `name`. Caso contrario, modal "QR Inválido".
- [3] Validación de sesión activa: si `attendanceActive` es falso, la lectura se descarta silenciosamente.
- [4] Validación de horario:
 - Si no hay `ClassSchedule` para el día → "Sin Horario"
 - Si `hora actual < startTime` → "Fuera de Horario"
 - Si `hora actual > endTime` → "Clase Finalizada"
- [5] Determinación de estado:
 - `hora actual <= maxLateTime` → `status = 'present'`
 - `hora actual > maxLateTime` → `status = 'late'`
- [6] Pre-check anti-duplicados: query por `studentId` + `date`. Si existe, modal "Ya Registrado" y se anade `exitAt` si corresponde.
- [7] Escritura atómica: `Firestore Transaction` sobre `classrooms/{id}/attendance/{computedId}` con:
 - `get(ref)` -> si existe, `throw 'YA_REGISTRADO'`
 - `set(ref, { studentId, studentName, classroomId, sessionId, date, status, timestamp: serverTimestamp(), entryAt: serverTimestamp() })`
- [8] Resultado UI: modal de éxito + actualización de `lastScannedCard`
- [9] Trigger `Firestore`: `onDocumentCreated` dispara la notificación Telegram

10.3 Registro de salida

Si en el paso 6 se detecta un registro existente sin `exitAt`, el repositorio `attendance_repository.dart` agrega el campo `exitAt = serverTimestamp()` mediante una operación de actualización. El resultado retornado al cliente es de tipo `QrScanResultType.exitRegistered`. Si el registro ya tiene `exitAt`, el resultado es `QrScanResultType.exitAlreadyRegistered`.

10.4 Sesión de asistencia

El registro QR está condicionado a la existencia de una sesión activa, representada en la colección `attendance_sessions`. El docente inicia la sesión explícitamente desde `ClassroomDetailScreen`, lo que crea un documento con `isActive: true`. Cada registro generado en la sesión incluye el `sessionId` correspondiente. El cierre de la sesión actualiza el documento con `endTime` y `attendanceCount` calculado.

11 Integración Telegram

11.1 Configuración del bot

El token del bot se obtiene de la variable de entorno `TELEGRAM_BOT_TOKEN`, con un valor de respaldo definido en `telegram.ts:15`. El nombre de usuario del bot se obtiene en tiempo de ejecución mediante la API `getMe` de Telegram y se cachea en memoria por instancia. Si la consulta falla, se utiliza la variable `TELEGRAM_BOT_USERNAME` o el fallback `'mi_bot_asistencia'`.

11.2 Generación de código de vinculación

La función callable `createTelegramActivationLink` ejecuta:

1. Validación de autenticación (`request.auth.uid` debe existir).
2. Validación de parámetro `studentId`.
3. Verificación de permisos: solo admin o docente con acceso al aula del estudiante puede generar el código.
4. Invalidación de códigos activos previos del mismo estudiante (`invalidatePreviousActivationCodes`).
5. Generación de un código aleatorio de seis dígitos: `Math.floor(100000 + Math.random() * 900000)`.
6. Persistencia en la colección `activation_codes` con `expiresAt = now + 24h`.
7. Retorno al cliente del código, el enlace `https://t.me/{botUsername}?start={code}` y un mensaje formateado para WhatsApp.

11.3 Normalización de teléfonos

El helper `toE164Peru()` normaliza los teléfonos al formato canónico:

```
toDigits(phone):      elimina todos los caracteres no numericos
toE164Peru(phone):    si empieza con '51' -> '+{phone}'
                      si tiene 9 digitos -> '+51{phone}' (movil peruano)
                      caso contrario -> '+{phone}'
```

Esta normalización permite la deduplicación de teléfonos almacenados en distintos formatos y garantiza compatibilidad con la API de Telegram.

11.4 Trigger de notificación automática

El trigger `processAttendanceEvent` reacciona a escrituras en la subcolección de asistencia. El procesamiento se delega al trigger de `attendance_events` cuando el documento tiene `eventDriven: true` y `source: 'attendance_event'`, evitando notificaciones duplicadas.

El flujo de notificación comprende:

1. Lectura del documento de asistencia creado o modificado.
2. Filtrado por status: solo se notifica para `['present', 'late', 'presente', 'tardanza', 'tarde']`.

3. Resolución del estudiante: búsqueda primero en `students/{id}` y, si no existe, en `users/{id}` (compatibilidad legacy).
4. Resolución del teléfono del apoderado: se admiten los campos `parentPhone`, `telefonoApoderado`, `telefonoPadre`, `parent_phone`.
5. Normalización E.164 del teléfono.
6. Resolución del `chatId` del apoderado vinculado.
7. Deduplicación por día mediante `getDayKeyLima(date)` (zona horaria America/Lima).
8. Formato del mensaje en español peruano (`locale es-PE`, `timezone America/Lima`).
9. Envío mediante `axios.post('https://api.telegram.org/bot{TOKEN}/sendMessage', ...)`.
10. Registro del evento en `telegram_events` para diagnóstico (append-only).

11.5 Códigos de activación

La colección `activation_codes` mantiene los códigos generados con la siguiente estructura representativa:

```
{
  code: '123456',
  studentId: string,
  studentName: string,
  parentPhone: string,
  parentPhoneDigits: string,
  parentPhoneE164: string,
  createdAt: Date,
  used: boolean,
  expiresAt: Date, // createdAt + 24h
  generatedByUid: string,
  source: 'manual-link' | 'auto-create',
  usedAt?: Date,
  deactivatedBy?: 'regenerated' | 'expired'
}
```

12 Sincronización Entre Colecciones

12.1 Doble esquema de almacenamiento

El sistema mantiene dos representaciones de los registros de asistencia:

1. **Esquema canónico:** `classrooms/{classroomId}/attendance/{attendanceId}` — subcolección bajo el aula. Es el destino principal de las operaciones de escritura del cliente.
2. **Esquema mirror:** `attendance/{studentId}_{dateKey}` — colección raíz con clave compuesta. Optimizada para queries globales y compatibilidad con vistas administrativas.

La sincronización entre ambos esquemas se realiza mediante un trigger Firestore que reacciona a escrituras en el esquema canónico.

12.2 Trigger `syncClassroomAttendanceToRoot`

Definido en `attendance-sync.ts:47-91`, este trigger se activa con `onDocumentWritten('classrooms/{classroomId}/attendance/{docId}')`. Su comportamiento es:

1. Si la escritura es una eliminación (`after.exists === false`), no realiza ninguna acción.
2. Si el documento carece de `studentId`, registra una advertencia y termina.
3. Calcula `dateKey` desde el campo `date`, `timestamp` o `entryAt` mediante `dayKeyFromAny()`.
4. Calcula el `documentId` raíz como `{studentId}_{dateKey}`.
5. Ejecuta un `set` con `{ merge: true }` sobre la colección raíz, mapeando los campos relevantes y normalizando el `status` a la notación legacy (`toLegacyStatus()`).

12.3 Conversión de status

El helper `toLegacyStatus()` realiza el siguiente mapeo:

Entrada	Salida
'presente' o 'present'	'present'
'tarde' o 'late'	'late'
'ausente' o 'absent'	'absent'
cualquier otro valor	'present' (fallback conservador)

12.4 Idempotencia

El uso de `set({ merge: true })` con un `documentId` determinístico garantiza la idempotencia del trigger. Múltiples ejecuciones por el mismo evento no producen registros duplicados ni anomalías en el estado final del documento.

13 Generación De Reportes

13.1 Fuentes de datos

La pantalla `TeacherReportsScreen` obtiene los datos mediante una secuencia que privilegia las funciones callable y emplea un fallback directo sobre Firestore en caso de error:

```
[1] PRIMARIO: Cloud Function exportReportData(  
  { classroomId, startDate, endDate, format: 'structured' })  
  Retorna: { metadata, summary, studentSummaries, attendances, students }  
  
[2] FALLBACK: collection('attendances').where('classroomId', '==', id)  
  Filtrado por fecha en memoria.  
  Nota: el nombre de coleccion en el fallback corresponde a un  
  esquema legacy que puede estar vacio. La coleccion canonica es  
  classrooms/{id}/attendance.
```

13.2 Generación PDF (modelo SIAGIE)

La generación del reporte PDF utiliza el paquete `pdf` (3.11.1) y se ejecuta on-device. El documento se compone como `pw.Document` con `pw.MultiPage`, formato A4 horizontal y margen uniforme de 15 puntos.

La tabla principal incluye una columna fija para el número correlativo, una columna flexible para apellidos y nombres y una columna por día del mes (de 1 a 28, 30 o 31 según el mes). Cada celda se rellena con el símbolo correspondiente al status: V para presente, T para tardanza, F para falta y J para justificada. Una leyenda al pie del documento explica la nomenclatura. La sección final incluye espacios para la firma del docente y del director.

El archivo generado se entrega al sistema operativo mediante `Printing.sharePdf()` y queda disponible para guardarlo, enviarlo por mensajería o imprimirlo.

13.3 Generación Excel (formato UGEL 06)

La generación Excel utiliza el paquete `excel` (4.0.6) y produce un archivo cuyo encabezado principal se compone con merge de celdas, formato bold blanco sobre fondo #1F4E78. La fila de cabecera contiene los días del mes y el cuerpo se rellena con símbolos coloreados:

- Presente: relleno #00B050 (verde).
- Tardanza: relleno #FFC000 (naranja).
- Falta: relleno #FF0000 (rojo).
- Justificada: relleno #0070C0 (azul).

Los anchos de columna se calibran para impresión en formato A4. La distribución del archivo se realiza mediante `Share.shareXFiles([XFile(path)], text: ...)`.

13.4 Métricas calculadas

Metrica	Calculo
Asistencia promedio	$(\text{present} + \text{late}) / \text{total} * 100$
Ausentismo cronico	numero de estudiantes con absent / total $\geq 20\%$
Tendencia semanal	porcentaje de asistencia por semana del mes (cuatro periodos)
Total de sesiones	dias distintos con al menos un registro de asistencia

14 Analisis Con Ia

14.1 Pipeline en tres pasos

La generacion del analisis institucional invoca tres funciones callable en secuencia:

```
[1] exportReportData({ classroomId, startDate, endDate, format })
    -> Snapshot estructurado del periodo

[2] getAttendanceReportData({ classroomId, startDate, endDate })
    -> Datos consolidados listos para el prompt

[3] generateReportWithAI({ classroomId, startDate, endDate, attendanceData })
    -> { summary, patterns, recommendations }
```

14.2 Modelo y proveedor

El analisis utiliza la SDK `@google-cloud/vertexai` con el modelo `gemini-pro` segun documenta `database_diagram.dbml:177` y la tabla `ai_reports`. La invocacion se encapsula en una Cloud Function callable para evitar exponer credenciales en el cliente movil.

14.3 Estructura del resultado

El cliente Flutter recibe un objeto con tres campos:

- `summary` (string): resumen narrativo del periodo.
- `patterns` (lista de strings): situaciones recurrentes identificadas.
- `recommendations` (lista de strings): sugerencias operativas accionables.

El cliente renderiza estos tres bloques en la pestana de Analisis IA del modulo de reportes. El procesamiento toma entre 10 y 30 segundos en condiciones normales y depende de la latencia de la red y de la disponibilidad del servicio de Vertex AI.

14.4 Latencia artificial

El cliente introduce una espera artificial de aproximadamente dos segundos previa al inicio del pipeline (`Future.delayed(Duration(seconds: 2))`) con el objetivo de asegurar la propagacion de las ultimas escrituras en Firestore antes de invocar el primer paso. Esta espera puede sustituirse a futuro por un mecanismo de polling de estado real.

15 Configuración Y Despliegue

15.1 firebase.json

El archivo de configuración principal declara:

- **Firestore:** base de datos (default) en region us-central1, reglas en `firestore.rules`, índices en `firestore.indexes.json`.
- **Functions:** codebase default en directorio `functions`, con `predeploy npm --prefix functions run build` para compilar TypeScript antes del despliegue.
- **Emulators:** configurados para `auth: 9099`, `functions: 5001`, `firestore: 8080`, `database: 9000` y la UI de emuladores habilitada.

15.2 Comandos de operacion

Comando	Efecto
<code>npm --prefix functions run build</code>	Compila TypeScript a JavaScript
<code>npm --prefix functions run watch</code>	Compila en modo watch
<code>npm --prefix functions run serve</code>	Compila e inicia emuladores
<code>npm --prefix functions run deploy</code>	Compila y despliega solo las Functions
<code>firebase deploy --only functions:api</code>	Despliega unicamente la funcion HTTP
<code>firebase emulators:start</code>	Inicia todos los emuladores configurados
<code>firebase functions:log</code>	Consulta los logs de las funciones desplegadas

15.3 Flujo de despliegue recomendado

1. Validar lint y build local (`npm run lint`, `npm run build`).
2. Ejecutar pruebas locales contra emuladores si aplica.
3. Verificar `firestore.indexes.json` (la creación de índices nuevos puede tomar minutos en producción).
4. Verificar `firestore.rules` con el simulador de reglas.
5. Desplegar Functions: `npm run deploy`.
6. Verificar logs y comportamiento postdespliegue.
7. Validar la disponibilidad del bot de Telegram y la conectividad con Vertex AI.

16 Variables De Entorno

16.1 Variables del backend

Variable	Uso	Obligatoriedad
TELEGRAM_BOT_TOKEN	Token del bot de Telegram para envío de mensajes	Recomendado
TELEGRAM_BOT_USERNAME	Username del bot como fallback si falla getMe	Opcional
AUTH_PASSWORD_RESET_URL	URL de retorno para recuperacion de contraseña	Opcional
CORS_ALLOWED_ORIGINS	Origenes autorizados para CORS en produccion	Recomendado
NODE_ENV	Controla la disponibilidad de endpoints de setup	Recomendado

16.2 Notas operativas

El archivo `telegram.ts:15` declara un valor de fallback hardcoded para `TELEGRAM_BOT_TOKEN`. Esta practica es aceptable para entornos de desarrollo pero debe corregirse antes de un despliegue en produccion definitiva, sustituyendo el fallback por un fallo controlado que requiera la variable de entorno explicita.

16.3 Secretos del cliente movil

El cliente movil requiere los archivos:

- `asistencias_app/android/app/google-services.json` (Android)
- `asistencias_app/ios/Runner/GoogleService-Info.plist` (iOS)

Ambos archivos contienen claves de proyecto Firebase y deben mantenerse fuera del repositorio publico. El archivo `.gitignore` debe excluirlos explicitamente.

17 Entorno De Desarrollo

17.1 Requisitos minimos

Componente	Version minima
Node.js	22
npm	10.8
Flutter SDK	3.9
Firebase CLI	actualizada (npm install -g firebase-tools)
Android Studio o Xcode	para construccion del cliente movil

17.2 Setup inicial

```
# Backend
cd functions
npm install
npm run build

# Frontend
cd ../asistencias_app
flutter pub get
flutterfire configure # solo la primera vez
flutter run
```

17.3 Emulacion local

La emulacion completa del entorno se realiza con `firebase emulators:start`. La UI de emuladores en `http://localhost:4000` permite inspeccionar Firestore, ejecutar funciones manualmente y administrar la autenticacion. Para que el cliente Flutter apunte a los emuladores en lugar de produccion, debe configurarse explicitamente la conexion mediante `FirebaseAuth.instance.useAuthEmulator()` y equivalentes para Firestore y Functions.

17.4 Validacion de cambios

Antes de fusionar cambios a la rama principal se recomienda:

- Ejecutar `npm run lint` y `npm run build` en `functions/`.
- Verificar que no se hayan introducido nuevos modelos sin migrar los existentes.
- Verificar que las nuevas queries esten respaldadas por indices en `firestore.indexes.json`.
- Probar el flujo QR completo en el emulador.
- Validar que las reglas de seguridad continuan otorgando y denegando acceso segun lo esperado.

18 Hallazgos Arquitectónicos

Durante el análisis del código fuente se identificaron las siguientes situaciones que merecen documentación explícita y, eventualmente, intervención planificada.

18.1 Coexistencia de versiones de modelos del frontend

El directorio `asistencias_app/lib/models/` contiene tres archivos para la misma entidad de asistencia: `attendance_model.dart`, `attendance_model_new.dart` y `attendance_models.dart`. La misma situación se presenta para `classroom` y `student` con dos archivos cada uno. Esta coexistencia indica una migración en curso entre esquemas. Se recomienda planificar la consolidación bajo una sola versión canónica.

18.2 Coexistencia de versiones del servicio administrativo

El directorio `asistencias_app/lib/services/` contiene `admin_service.dart`, `admin_service_new.dart` y `admin_service_final.dart`. La denominación sugiere que `admin_service_final.dart` es la versión canónica vigente, pero la persistencia de los otros archivos puede inducir a error en mantenimiento. Se recomienda eliminar los archivos obsoletos.

18.3 Coexistencia de notación en colecciones y estados

El sistema acepta indistintamente la notación canónica en inglés (`'present'`, `'late'`, `'absent'`) y la notación legacy en español (`'presente'`, `'tarde'`, `'ausente'`), tanto en las reglas de Firestore como en el helper `toLegacyStatus()` del backend. La coexistencia es funcional pero introduce complejidad en las consultas y aumenta la superficie de error. Se recomienda planificar la normalización completa hacia la notación canónica.

18.4 Coexistencia de esquemas legacy en Firestore

Las colecciones `salones` y `asistencias` (en español) coexisten con `classrooms` y `attendance` (en inglés). Ambas mantienen índices declarados y reglas de seguridad. La presencia simultánea responde a una migración documentada en el directorio `SALONES_API_DOCS.md`. Se recomienda planificar la deprecación progresiva del esquema legacy.

18.5 Token de Telegram con fallback hardcoded

El archivo `telegram.ts:15` declara un valor de respaldo hardcoded para el token del bot:

```
const BOT_TOKEN = process.env.TELEGRAM_BOT_TOKEN ||  
  '8305613209:AAFxld-nM5Qwe5Rs1TTDEbyXH0du2Vg_NQw';
```

Aunque el patrón es común en desarrollo, el token de producción no debe vivir en el código fuente. Se recomienda sustituir el fallback por un error explícito si la variable no está definida.

18.6 Lista de administradores por email hardcoded

Las reglas de Firestore (`firestore.rules:20`) mantienen una lista blanca de administradores por correo electrónico:

```
function isAdminEmail() {  
  return request.auth.token.email in  
    ['admin@asistencias.com', 'luisangelfabian10@gmail.com'];  
}
```

Este patron permite la operacion administrativa de respaldo en caso de que el sistema de roles falle, pero requiere mantenimiento manual y revision de auditoria. Se recomienda evaluar su sustitucion por un mecanismo de roles centralizado.

18.7 Fallback de reportes con coleccion incorrecta

En `teacher_reports_screen.dart` la rama de fallback ante fallo de Cloud Function consulta `collection('attendances')` (en espanol, plural legacy), mientras que la coleccion canonica es `classrooms/{id}/attendance` o el mirror raiz `attendance`. Esto provoca que el fallback produzca resultados vacios. Se recomienda corregir el nombre de la coleccion.

18.8 Doble pantalla de escaneo QR

El frontend incluye dos pantallas con funcionalidad de escaneo QR: `quick_qr_attendance_screen.dart` y el escaner embebido en `classroom_detail_screen.dart`. La duplicidad puede inducir a confusion en la operacion diaria y dificulta el mantenimiento. Se recomienda definir una unica pantalla canonica.

18.9 Rol alumno modelado pero bloqueado

El rol 'alumno' esta definido en el tipo de la propiedad `role` de `Request.user` (`index.ts:17`) y en el modelo de datos, pero todas las solicitudes con este rol son rechazadas por el middleware `blockStudentAccess`. Esto documenta el estado beta del sistema. La habilitacion futura del rol requerira la implementacion de pantallas dedicadas en el cliente movil y la revision de las reglas de seguridad.

18.10 Espera artificial en analisis IA

El pipeline de analisis IA introduce una espera artificial de dos segundos antes de invocar la primera funcion (`Future.delayed(Duration(seconds: 2))`) para asegurar la propagacion de escrituras en Firestore. Esta espera puede sustituirse por una verificacion explicita del estado del documento, eliminando latencia innecesaria en casos en los que las escrituras ya estan propagadas.

19 Riesgos Técnicos Identificados

19.1 Riesgos de configuración

ID	Riesgo	Impacto
RT01	Indice Firestore faltante para nueva query	Falla la query con error de indice ausente
RT02	Variable TELEGRAM_BOT_TOKEN no establecida en produccion	Uso del token hardcoded de desarrollo
RT03	Schedule del aula sin configurar	Escaneo QR bloqueado para el aula
RT04	maxInstances: 10 insuficiente en hora punta	Escalado limitado bajo carga concurrente alta

19.2 Riesgos operativos

ID	Riesgo	Impacto
RT05	Token de Telegram expira o se invalida	Suspension de notificaciones a apoderados
RT06	Trigger syncClassroomAttendanceToRoot falla	Datos desactualizados en vistas globales
RT07	Vertex AI no disponible	Bloqueo del modulo de analisis IA
RT08	Latencia de cold start en Cloud Functions	Primera llamada lenta tras periodos de inactividad

19.3 Riesgos de datos

ID	Riesgo	Impacto
RT09	Coexistencia de modelos legacy	Inconsistencias en reportes que mezclan ambos esquemas
RT10	Mezcla de notacion ES/EN en status	Errores de filtrado y agrupacion en reportes
RT11	Telefonos de apoderado sin normalizar	Falla de vinculacion Telegram
RT12	Documentos de asistencia sin studentId	El trigger de sincronizacion omite el registro

19.4 Riesgos de seguridad

ID	Riesgo	Impacto
RT13	CORS abierto a * en produccion	Solicitudes desde origenes no autorizados
RT14	Endpoint /auth/setup-admin activo en produccion	Creacion de cuentas administrativas sin control
RT15	Lista de admin por email hardcoded en reglas	Dependencia de despliegue de reglas para revocacion

20 Recomendaciones De Evolucion

20.1 Consolidacion del modelo de datos

- Unificar los modelos `attendance_model.dart` / `_new` / `_models` en una unica version canonica.
- Eliminar `admin_service.dart` y `admin_service_new.dart`, manteniendo unicamente `admin_service_final.dart`.
- Planificar la migracion completa de `'docente'` / `'teacher'` hacia un unico valor canonico (`'docente'`).
- Planificar la migracion completa de notacion de status hacia la version canonica en ingles.

20.2 Deprecacion de esquemas legacy

- Definir un plan de migracion de los clientes que utilizan endpoints `/salones` y `/asistencias` hacia los nuevos endpoints.
- Marcar como deprecados los indices y reglas asociadas a las colecciones legacy.
- Eliminar las colecciones legacy una vez completada la migracion.

20.3 Endurecimiento de seguridad

- Sustituir el CORS abierto por una lista explicita de origenes autorizados en produccion.
- Restringir o eliminar el endpoint `/auth/setup-admin` en despliegues productivos.
- Sustituir la lista hardcoded de administradores por email por un mecanismo basado en roles persistentes.
- Sustituir el fallback hardcoded del token de Telegram por un fallo controlado.

20.4 Mejoras de fiabilidad

- Implementar metricas y alertas para los triggers `syncClassroomAttendanceToRoot` y de notificacion Telegram.
- Implementar reintentos automaticos con backoff en `processAttendanceEvent` ante fallos transitorios de la API de Telegram.
- Definir un mecanismo de salud (`/health`) que verifique conectividad con Telegram y con Vertex AI.

20.5 Mejoras de performance

- Sustituir la consulta de correcciones que descarga toda la coleccion por una query con filtro de fecha en servidor y paginacion por cursor.
- Sustituir la espera artificial de dos segundos en el pipeline IA por una verificacion de estado real.
- Considerar el incremento de `maxInstances` para Functions identificadas como puntos calientes.

20.6 Funcionalidades futuras

- Habilitar el rol 'alumno' con pantallas dedicadas para consulta personal de asistencia.
- Incorporar un portal para apoderados con vista de historial.
- Incorporar reportes globales para el rol administrador con consolidación por aula y por periodo.
- Incorporar un dashboard de monitoreo de sesiones activas en tiempo real para el rol administrador.

21 Glosario Técnico

Callable function

Función de Cloud Functions invocable directamente desde el SDK cliente, con autenticación implícita. Se distingue de las funciones HTTP por su interfaz tipada y por la inyección automática del contexto de autenticación.

Custom claims

Atributos asignados al token de Firebase Auth que el cliente o las reglas de seguridad pueden leer sin requerir lecturas adicionales a la base de datos.

dateKey

Cadena con formato YYYY-MM-DD utilizada como componente de claves compuestas y para deduplicación diaria.

E.164

Estandar internacional de numeración telefónica. La normalización peruana adoptada por el sistema produce cadenas con prefijo +51.

Firestore Transaction

Operación atómica sobre Firestore que combina lecturas y escrituras dentro de una unidad consistente. Utilizada para garantizar la unicidad del registro de asistencia diaria.

ID Token

Token JWT emitido por Firebase Authentication, válido por una hora y refrescado automáticamente por el SDK cliente.

Mirror collection

Colección replicada de manera automática a partir de un trigger Firestore. En este sistema, la colección raíz attendance es un mirror de la subcolección classrooms/{id}/attendance.

Schedule

Objeto anidado dentro del documento de aula que declara, por día de semana, los tiempos startTime, endTime y maxLateTime.

Trigger Firestore

Función de Cloud Functions invocada automáticamente ante eventos de creación, escritura o eliminación de documentos en Firestore.

Vertex AI

Plataforma de Google Cloud para servicios de inteligencia artificial. Utilizada por el sistema mediante el modelo Gemini Pro para la generación de análisis institucional de asistencia.

22 Referencias

22.1 Archivos clave del repositorio

Ruta	Contenido
firebase.json	Configuración del proyecto Firebase
firestore.rules	Reglas de seguridad declarativas
firestore.indexes.json	Indices compuestos
functions/package.json	Dependencias y scripts del backend
functions/src/index.ts	Aplicación Express y configuración global
functions/src/telegram.ts	Lógica del bot de Telegram
functions/src/attendance-sync.ts	Trigger de sincronización
functions/src/modules/	Módulos REST en Clean Architecture
asistencias_app/pubspec.yaml	Dependencias del cliente Flutter
asistencias_app/lib/main.dart	Entry point del cliente
asistencias_app/lib/screens/teacher/classrooms/classroom_detail_screen.dart	Pantalla crítica del flujo QR
database_diagram.dbml	Modelo de datos en notación DBML

22.2 Documentos institucionales relacionados

- DOCUMENTACION_MAESTRA.md — Referencia estructural completa del sistema.
- MANUAL_DOCENTE.md — Manual operativo del rol docente.
- MANUAL_ADMINISTRADOR.md — Manual operativo del rol administrador.
- SALONES_API_DOCS.md — Documentación del esquema legacy de salones.

22.3 Documentación externa relevante

- Documentación oficial de Firebase Authentication.
- Documentación oficial de Cloud Firestore (reglas, transacciones, índices).
- Documentación oficial de Cloud Functions v2.
- Documentación oficial de la API de Telegram Bots.
- Documentación oficial de Vertex AI Gemini Pro.

Sistema Inteligente de Asistencia QR

Manual Tecnico — Documentacion de Arquitectura | Version 1.0 | 18 de mayo de 2026

Institucion Educativa N.º 1254 Maria Reiche Newman

UGEL 06 — Ate Vitarte, Lima | Ano lectivo 2026

Para consultas tecnicas, dirijase al responsable de sistemas de la institucion.
